Output-Maximizing Long-Context Programming: Why Agentic Coding Workflows Scale Poorly and What to Do Instead

Jason Viviers

March 11, 2026

Abstract

Current large language model (LLM) development tooling is dominated by agentic, iterative workflows that emphasize small outputs, frequent context rehydration, and substantial invisible reasoning overhead. This paper argues that these workflows represent an artifact of historical output constraints rather than an inherent requirement of intelligent systems. We introduce Output-Maximizing Long-Context Programming (OMLCP or Output Maxing in short), a paradigm that exploits large output context windows to generate coherent, large-scale artifacts in single or near-single constrained executions. Through formal token economics analysis, illustrative calculations, and field evidence from three case studies, we demonstrate that agentic workflows exhibit severe compounding inefficiencies driven primarily by non-output overhead, especially repeated context rehydration. In contrast, OMLCP minimizes reasoning overhead, preserves global coherence, and can reduce total token cost by one to two orders of magnitude while improving artifact quality and architectural consistency. We validate this approach with a field case study generating 14,431 lines of code at $0.58 total cost, achieving a 26-52× efficiency advantage over comparable agentic approaches.

1 Introduction

The landscape of AI-assisted software development has converged on a particular pattern: iterative, tool-augmented interaction in which large language models perform sequences of small actions, each requiring fresh reasoning over repeatedly supplied context. This approach manifests in chat-driven development, copilot-style file editing, autonomous agent frameworks, and planner-executor loops. While demonstrably effective for exploratory work, task-based agentic work and incremental modifications, this paradigm scales poorly as artifact size increases.

This scaling failure is not fundamentally a limitation of model intelligence or capability. Rather, it represents a structural inefficiency embedded in the architecture of contemporary development workflows. The dominance of agentic patterns reflects reasonable historical constraints, specifically, the limited output windows and expensive context handling that characterized earlier generations of language models. As models evolved to support dramatically larger output contexts, the tooling ecosystem has largely failed to capitalize on this capabil-

ity shift.

Modern frontier models now support unprecedented output capacities: GPT-5.4 supports a 1,050,000-token context window with 128,000 max output tokens [3], GPT-5.2 offers 128,000 output tokens with a 400,000 token context window [2], Claude Sonnet 4.6 provides 64,000 output tokens with a 200,000-token context window (1 million in beta) [4], and Gemini 3.1 Pro Preview delivers 65,536 output tokens with a 1,048,576 token input window [5]. Yet most development environments continue to optimize for short responses and frequent interaction, artificially constraining what these models can produce.

To reduce model-version obsolescence, we also frame this paper in capability tiers rather than product names. Tier A systems provide approximately 1M context with around 100k+ output capacity; Tier B systems provide roughly 400k to 1M context with 64k to 128k output; Tier C systems provide shorter context and/or output budgets where decomposition pressure is higher. Specific model names above should therefore be interpreted as dated examples (as of March 2026), while the central claims target capability classes.

This paper introduces and formalizes Output-Maximizing Long-Context Programming (OMLCP or Output Maxing in short), an alternative paradigm designed to align workflow architecture with contemporary model capabilities. We demonstrate that this approach yields substantial improvements across multiple dimensions: reduced computational cost, enhanced global coherence, improved predictability, and more favorable scaling characteristics.

To keep claims interpretable, we separate three epistemic tiers from the outset: analytical implications from the cost model, observed field evidence from deployments, and forward-looking hypotheses about future model/tooling trajectories. This structure is intentional and is used throughout the paper to distinguish what is measured, what is modeled, and what is projected.

Table 1: Capability tiers used throughout this paper.

Tier	Capability profile	Workflow implication
A	∼1M context and ≥100k output	Strong fit for near-single-pass OMLCP generation
B	400k–1M context and 64k–128k output	OMLCP remains effective with bounded continuation
C	Lower context and/or output ceilings	Higher decomposition pressure and more agentic looping

2 Background: The Rise of Agentic Workflows

2.1 Historical Context and Constraints

Early production coding workflows were shaped by tighter model limits. GPT-3.5 Turbo, widely used as a baseline in early modern tooling, supports a 16,385-token context window and 4,096 maximum output tokens [1]. Under these con-

ditions, substantial programming tasks still required decomposition into smaller pieces. As context windows expanded to 128,000 tokens and beyond, the fundamental interaction pattern persisted, shaped by user interface conventions, established development patterns, and API design choices.

2.2 The Agentic Paradigm

We define Agentic Iterative Programming (AIP) as any workflow in which an LLM performs a sequence of actions, each requiring fresh reasoning over re-supplied context. This encompasses:

- Chat-driven development with incremental software construction
- File-by-file editing systems with targeted modifications
- Tool-augmented agents with plan-execute-validate loops
- Planner-executor architectures with subtask decomposition
- Autonomous coding systems with trial-and-error iteration

The unifying characteristic is decomposition: large tasks are broken into many small steps, each involving a complete cycle of context ingestion, reasoning, action selection, and output generation.

2.3 The Hidden Cost Structure

Each agentic step incurs three token categories: input (context required for orientation), reasoning (internal deliberation), and output (visible artifacts). Recent software-engineering-specific measurement studies show that context transfer and coordination overhead are typically the dominant cost center. Tokenomics reports aggregate shares of 53.9% input, 24.4% output, and 21.6% reasoning across multi-agent software trajectories, and labels the resulting repeated inter-agent context passing as a *communication tax* [6].

Complementary evidence from a preprint study of OpenHands agent trajectories on SWE-bench (withdrawn from ICLR 2026; findings remain publicly available) reports that input tokens dominate total consumption and cost in agentic coding workflows even with token caching, with substantial run-to-run variance (up to 10×) that undermines budget predictability [7]. SWE-Pruner independently reports that read-type actions (for example `cat`, `grep`, and `head`) account for 76.1% of token costs in Mini SWE-Agent trajectories on SWE-Bench Verified [8].

Taken together, these studies indicate that non-output tokens are structurally dominant in agentic coding cost profiles, with context rehydration as the primary driver. The Tokenomics aggregate implies a 75.5% non-output baseline, while read/coordination-heavy trajectories can push non-output share into the 90–98% range in practice [6–8].

2.4 Compounding Effects

Consider a realistic scenario: a project requiring 100 high-level prompts, each triggering 10 internal agent steps, yielding 1,000 total model invocations. At 3,000 tokens per invocation, this consumes approximately 3 million tokens total. Using the Tokenomics aggregate split (53.9/24.4/21.6), this corresponds to about 1.617 million input tokens, 732,000 output to-

kens, and 648,000 reasoning tokens [6].

Even under this comparatively output-friendlier profile, non-output tokens still consume approximately 2.265 million tokens for only 732,000 visible output tokens. Real-world projects can scale worse: debugging loops, weak specifications, and context rehydration failures introduce repeated rereads and rework, while OpenHands reports up to 10× run variance on similar benchmarks [7]. We have documented cases where agentic workflows consumed 3 million tokens to produce artifacts generatable in under 50,000 tokens using OMLCP, a 60× cost difference.

3 Output-Maximizing Long-Context Programming

OMLCP represents an architectural inversion of the agentic paradigm. Rather than minimizing output to enable frequent interaction, it minimizes interaction to enable coherent output.

3.1 Core Principles

OMLCP is founded on four essential principles:

Comprehensive Upfront Planning Requirements, architecture, and structural decisions are thoroughly established before generation begins. This planning phase may involve AI assistance but produces explicit specifications rather than distributed intent.

Constrained Input Context Generation prompts contain only information directly relevant to the task. Unnecessary context is aggressively eliminated. Even for large projects, input contexts typically remain under 30,000 tokens through careful curation.

Maximized Coherent Output The model is instructed to produce the entire artifact in a single generation pass. Output contexts of 50,000 to 128,000 tokens are routine, fully exploiting modern model capabilities. In long generations near output limits, the model can be explicitly instructed to stop with an incomplete but structurally valid stream rather than emit stubs or summary placeholders. The same conversation ID can then be used for bounded continuation, preserving high coherence while extending generation across turns within the model's maximum context constraints.

Minimal Reasoning Verbosity Model reasoning is disabled or minimized during generation. The planning phase replaces most reasoning requirements, allowing the model to focus on producing structured output.

3.2 Workflow Architecture

A typical OMLCP workflow proceeds through five distinct phases:

1. **Requirements and Specification:** Articulate goals, constraints, and structural requirements through iterative refinement, producing a complete specification document.

2. **Architecture and Planning:** Establish high-level architecture including component structure, data flow, interface contracts, naming conventions, and dependencies.

3. **Generation Prompt Construction:** Construct a single comprehensive prompt

incorporating the complete specification, architectural decisions, explicit output format requirements, and clear boundaries.

4. **Long-Form Generation:** Generate the complete artifact in a single streaming response with no mid-generation corrections or interventions; when output ceilings are reached, stop at a valid structural boundary and continue deterministically in the same conversation trajectory.

5. **Deterministic Repair:** Handle necessary corrections through targeted edits using diff-based approaches rather than conversational iteration; this repair phase is distinct from bounded continuation, which extends the original generation trajectory.

3.3 Why This Works

OMLCP succeeds because it aligns with how frontier language models achieve coherent long-form output. When a model maintains unbroken attention across an entire artifact:

- Architectural decisions remain internally consistent
- Naming conventions stabilize naturally
- Implicit assumptions don't drift between calls
- Structural patterns repeat reliably
- Thematic or stylistic elements persist

This mirrors human cognition during focused work. The economic advantage follows directly: by replacing many reasoning-heavy cycles with one coherent generation, token consumption shifts from invisible overhead to visible output.

4 Token Economics: A Formal Analysis

To quantify the efficiency gains of OMLCP, we develop a formal model of token consumption across both paradigms.

4.1 Agentic Workflow Model

The following equations are a stylized cost model designed to show scaling behavior, not a claim that all agentic systems empirically exhibit these exact token shares.

Let:

L = total lines of code (or equivalent prose units)
T = average tokens per line
I = number of iterations required
C = context tokens per iteration
R = reasoning tokens per iteration
O = output tokens per iteration

Total cost for agentic approach:

$$\text{Cost}_{\text{agentic}} = I \times (C + R + O) \tag{1}$$

For analytic tractability, we use a stylized reasoning-heavy iterative profile (not a universal empirical average; see Section 2.3 for measured distributions):

$I \approx L/50$ (one iteration per 50 lines)
$C \approx 0.2L$ (20% of artifact size as context)
$R \approx 2280$ (high-overhead reasoning allowance)
$O \approx 120$ (low visible output per iteration)

This yields:

$$\begin{aligned}\text{Cost}_{\text{agentic}} &\approx \frac{L}{50} \times (0.2L + 2400) \\ &\approx 0.004L^2 + 48L \end{aligned} \tag{2}$$

The quadratic term reflects context inflation: as artifacts grow, maintaining context becomes increasingly expensive.

4.2 OMLCP Model

For OMLCP:

$$\text{Cost}_{\text{OMLCP}} = C_{\text{plan}} + L \times T \tag{3}$$

Where C_{plan} is fixed planning context (typically 20,000-30,000 tokens) and $L \times T$ is direct generation. With $T \approx 3$ tokens per line:

$$\text{Cost}_{\text{OMLCP}} \approx 25,000 + 3L \tag{4}$$

4.3 Comparative Analysis

For a 10,000 line project:

$$\begin{aligned}\text{Cost}_{\text{agentic}} &\approx 0.004(10,000)^2 + 48(10,000) \\ &\approx 880,000 \text{ tokens}\end{aligned}$$

$$\begin{aligned}\text{Cost}_{\text{OMLCP}} &\approx 25,000 + 3(10,000) \\ &= 55,000 \text{ tokens}\end{aligned}$$

Efficiency ratio: $880,000/55,000 = 16\times$

The advantage scales with project size. For a 30,000 line system:

$$\begin{aligned}\text{Cost}_{\text{agentic}} &\approx 5,040,000 \text{ tokens} \\ \text{Cost}_{\text{OMLCP}} &\approx 115,000 \text{ tokens} \\ \text{Efficiency ratio} :&\approx 44\times\end{aligned}$$

4.4 Sensitivity Analysis of Reasoning Share

The Section 4 equations are illustrative and should not be interpreted as universal constants. To test robustness, we vary the assumed reasoning share over a broad range (50% to 85%) while keeping other modeling assumptions fixed (including $C \approx 0.2L$ and an output floor of 120 tokens per iteration).

Let r be the reasoning share assumption for a 3,000-token reasoning budget proxy, so $R(r) = 3000r$. Then:

$$\text{Cost}_{\text{agentic}}(r) \approx \frac{L}{50}(0.2L + 3000r + 120) \tag{5}$$

$$\text{Cost}_{\text{agentic}}(r) \approx 0.004L^2 + (2.4 + 60r)L \tag{6}$$

For $L = 10,000$ and $\text{Cost}_{\text{OMLCP}} = 55,000$:

- $r = 0.50$: $\text{Cost}_{\text{agentic}} \approx 724,000$ ($\approx 13.2\times$ OMLCP)
- $r = 0.76$: $\text{Cost}_{\text{agentic}} \approx 880,000$ ($16\times$ OMLCP)
- $r = 0.85$: $\text{Cost}_{\text{agentic}} \approx 934,000$ ($\approx 17.0\times$ OMLCP)

For $L = 30,000$ and $\text{Cost}_{\text{OMLCP}} = 115,000$, the same range yields approximately $39.8\times$ to $45.2\times$ advantage. The conclusion is therefore stable across a wide reasoning-share interval: OMLCP maintains a large efficiency gap even when parameterized conservatively.

5 Coherence and Quality Analysis

Beyond economic efficiency, OMLCP offers substantial quality advantages stemming from global coherence preservation.

5.1 The Coherence Problem

Agentic workflows optimize locally. Common coherence failures include:

- **Naming drift:** Variables and functions acquire inconsistent names across sections
- **Architectural erosion:** Design patterns degrade as modifications accumulate
- **Thematic inconsistency:** Character voices or narrative tone drift in creative works
- **Assumption divergence:** Implicit assumptions about system behavior diverge
- **Interface misalignment:** API contracts evolve incompatibly across components

These failures represent entropy that accumulates when global context fragments across iterations.

This degradation pattern is now visible in benchmark data. SWE-EVO evaluates long-horizon software evolution tasks spanning an average of 21 files per issue; the top reported agent achieves 21% on SWE-EVO compared with 65% on single-issue SWE-Bench Verified tasks, indicating a sharp performance drop as coordination breadth and temporal depth increase [9].

5.2 Coherence Preservation in OMLCP

When a model maintains unbroken attention throughout artifact generation, these failure modes largely disappear. The model holds the entire emerging structure in working memory, allowing it to reference earlier decisions, maintain consistent naming, preserve architectural patterns, and ensure interface contracts match.

This reflects how attention mechanisms in transformer architectures work: the model can directly attend to earlier tokens in the output sequence, maintaining consistency through the attention mechanism itself rather than through external memory reconstruction.

6 Case Studies and Empirical Validation

6.1 Full-Stack Application

Project: A full-stack application with 14,431 lines of code, generated via a custom streaming code generator that creates live repositories from continuous output.

Planning Phase: Approximately 1 hour of upfront architectural planning and specification development, defining application structure, API contracts, component hierarchy, and deployment requirements.

Generation Phase: 20 minutes of continuous streaming generation, producing the entire application codebase in a single uninterrupted pass, with files and directories created in real-time.

Configuration and Integration: 1 hour of post-generation configuration, primarily connecting external services and setting environment variables. Only 1 small build error occurred, resolved in minutes through targeted editing.

Economic Analysis: Total cost: \$0.58 for 14,431 lines of production code (\$0.00004 per line, or 25,000 lines per dollar). A comparable agentic workflow could require approximately 289 iterations at 10,000 tokens each (2,890,000 total tokens), costing in the region of \$15-30 depending on the model and input/output ratios, representing a 26-52× cost advantage for OMLCP.

Quality Observations: The generated application worked immediately upon completion

of configuration. No architectural refactoring was required. All components integrated correctly on first attempt, demonstrating the coherence advantages predicted by OMLCP theory.

Time Analysis: Total time from conception to working application: 2 hours 20 minutes. Comparable agentic development typically requires multiple days.

6.2 Long-Form Fiction (Controlled Anecdote)

Project: A 45,000-word science fiction novella generated under two workflow conditions.

Design Controls: Both conditions used the same high-level outline, character set, world-building brief, and target length band. Prompt structure and acceptance criteria were kept as consistent as practical across conditions, with workflow style (iterative chapter sessions versus OMLCP long-form generation) as the primary intentional difference.

Iterative Condition: 89 chapter-by-chapter sessions, substantial character voice drift, plot inconsistencies requiring revision, and an estimated 15 million total tokens across drafting and correction.

OMLCP Condition: Single 58,000-token generation pass for the full novella, consistent character voice, and coherent resolution of major plot threads, with approximately 70,000 total tokens consumed including light cleanup. Estimated cost ratio: 214×.

Evaluation and Caveats: AI-assisted and manual qualitative reviews judged the OMLCP variant as more globally unified and better paced, but this remains an exploratory comparison rather than a blinded literary study. We therefore treat the fiction result as supportive directional evidence, not definitive causal proof.

6.3 Batch Marketing Website Rebuilds

Project: Controlled OMLCP test rebuilding 10 outdated marketing websites into enhanced sales-qualification pages.

Specification Pipeline: GPT-5.4 (thinking mode in ChatGPT) was used to analyze existing sites and generate contract-compliant technical specifications before implementation. The implementation stack was standardized across sites: Astro for frontend delivery, Cloudflare Turnstile for form protection, Cloudflare Functions for server logic, Cloudflare D1 for lightweight data storage, and Cloudflare R2 for media storage [3]. Image assets were specified upfront from available source material to reduce setup overhead, and dependency manifests were predeclared to reduce version drift.

Planning and Setup: Planning was parallelized and took approximately 15 minutes to gather contact information, branding constraints, industry context, and conversion-oriented content requirements. Per-site configuration and run validation averaged roughly 10 minutes, and this stage can be further automated.

Aggregate OMLCP Metrics (10 sites):

- Total cost: $6.2415 (average $0.62415 per site)
- Total tokens: 560,291 (average 56,029 per site)
- Average prompt tokens: 15,153; average completion tokens: 40,876
- Average completion-to-prompt ratio: 2.79×
- Summed model generation runtime: 81.99 minutes (8.20 minutes average per site)

Timing Interpretation: We report three timing dimensions separately to avoid ambiguity: planning time, summed model runtime, and parallel batch wall-clock time. The clearest 5-site parallel batch completed in 14 minutes 1 second wall-clock, while a separate overlapping 3-site batch completed in 11 minutes 55 seconds. The full span across all 10 logged runs was longer because jobs were executed in multiple batches and interleaved with other work.

Outcome: This test demonstrates that OMLCP can deliver high-output, low-marginal-cost batch rebuilds when specifications are standardized upfront. The outputs were strongly output-dominant and operationally practical for rapid deployment-oriented website modernization.

6.4 Methodological Notes and Reproducibility Protocol

The case studies above are intentionally practice-oriented and should be read as high-signal field evidence rather than controlled laboratory proof. To support independent validation, we propose a reproducible comparison protocol:

1. **Task Pairing:** Select matched tasks (e.g., CRUD API, microservice deployment, medium-scale refactor) with fixed acceptance tests and equivalent functional requirements.

2. **Workflow Conditions:** Run each task under two predefined conditions: (a) agentic iterative baseline and (b) OMLCP single-pass or near-single-pass generation.

3. **Model and Budget Controls:** Hold model family, temperature, and pricing assumptions constant where possible; report any unavoidable differences explicitly.

4. **Primary Metrics:** Report total tokens, reasoning-token share (if available), wall-clock time, iteration count, first-pass test pass rate, and post-generation defect count.

5. **Coherence Metrics:** Measure naming consistency, interface contract violations, architectural rule violations, and style entropy across files.

6. **Repair Accounting:** Separate deterministic repair tokens from initial generation tokens to avoid mixing generation quality with downstream maintenance.

This protocol makes OMLCP claims falsifiable and enables apples-to-apples benchmarking across teams, tools, and model generations.

In our current agentic comparison runs, the baseline environment was Windsurf. A practical limitation was token observability: Windsurf did not expose sufficiently granular per-step token telemetry for robust accounting of prompt, completion, and reasoning shares. We therefore treat current agentic token totals as conservative and plan follow-on investigations with telemetry-first tooling (for example, OpenCode) to capture per-turn token flow, cache effects, and tool-call overhead with audit-quality logs.

6.5 Implementation in Practice During Testing

To complement the theoretical model, our testing workflow implemented OMLCP as a spec-first generation pipeline rather than an iterative file-edit loop. In practical terms, each test run followed four stages:

1. **Upfront specification hardening:** Requirements, architecture, constraints, and acceptance expectations were consolidated before generation, so most ambiguity was resolved pre-inference.

2. **Protocol-constrained long-form generation:** The model was instructed to emit a structured multi-file output stream under explicit formatting and path-safety constraints, enabling deterministic parsing and repository materialization.

3. **Bounded continuation when necessary:** If output limits were reached, generation resumed through controlled continuation prompts that preserved the same output trajectory rather than reverting to conversational re-planning.

4. **Deterministic validation and repair:** Generated artifacts were validated against predefined checks, and any corrections were applied as targeted deterministic edits with explicit accounting of additional token and time cost.

This implementation choice is central to the observed results: it shifts effort from repeated in-loop reasoning to pre-generation design quality, reduces context rehydration overhead, and preserves global coherence across large outputs. Importantly, the testing protocol evaluates not just final correctness but also process efficiency, including iteration count, continuation frequency, and repair burden. We additionally track continuation success rate and post-continuation defect density to quantify whether multi-turn extension preserves architectural coherence in practice.

7 Positioning Relative to Adjacent Paradigms

OMLCP overlaps with several active research and industry themes, but remains distinct in workflow design.

7.1 Agentic Systems

ReAct and related agentic paradigms optimize for sequential tool use and iterative decomposition [15, 16]. OMLCP instead treats decomposition as a planning concern and concentrates implementation into one long coherent generation pass. The difference is architectural: agentic systems amortize uncertainty across many calls; OMLCP amortizes uncertainty into upfront specification quality.

7.2 Context Engineering

Context engineering has emerged as a practical discipline for selecting and ordering context in production LLM systems [19]. OMLCP can be viewed as a specialized regime within this broader discipline: aggressive context curation before generation, followed by maximal output continuity during generation.

7.3 Long-Context Reasoning Literature

Recent work explores prompt-only long-context reasoning and multi-agent approaches to long-context processing [17, 18]. These lines validate the importance of context strategy but primarily target reasoning and retrieval tasks. OMLCP contributes a software-production-specific framing with explicit economic and coherence objectives.

7.4 Practical Value of This Distinction

This positioning matters for adoption. Without a clear boundary, OMLCP can be misinterpreted as "just better prompting" or "another agent configuration." In practice, it is a workflow-level choice with different tooling requirements, developer skills, and performance envelopes.

8 Operational Trade-Offs and Hybrid Execution

This section makes explicit the mechanisms behind agentic coding systems and OMLCP, and clarifies where each paradigm is expected to dominate.

8.1 Mechanics of Agentic Coding Loops

Agentic systems follow a repeated reason-act-observe cycle, consistent with ReAct-style tool use and its descendants [15, 20, 21]. This iterative pattern is valuable under uncertainty because each tool result updates the model's belief state before the next decision. More broadly, reasoning-focused results such as self-consistency reinforce why multi-path or staged deliberation can help when search spaces are ambiguous [22].

8.2 Why Context Degrades in Practice

Even with large context windows, three degradation mechanisms remain central: token budget pressure, retrieval non-uniformity across long prompts (including lost-in-the-middle effects), and lossy compaction of earlier interaction traces [17, 23]. These mechanisms explain why long-running sessions can drift despite nominally large context limits.

8.3 OMLCP Operating Envelope

OMLCP changes the optimization target from iterative search to high-coherence production in one or few calls. It performs best when requirements are sufficiently specified before generation, and when success depends more on global consistency than on runtime discovery.

8.4 Decision Heuristic for Paradigm Selection

A practical heuristic is to choose OMLCP when scope is clear, architecture is constrained, and token efficiency is critical; choose agentic iteration when requirements are ambiguous, environment state is unknown, or runtime diagnostics drive progress. Prior SWE-agent evaluations and agentic literature support the value of iterative traces in uncertain tasks [15, 24].

9 Input Inflation and Tool-Driven Overhead

Modern tool-integrated systems, particularly those following the Model Context Protocol (MCP), introduce additional inefficiencies that OMLCP explicitly avoids.

9.1 The MCP Context Problem

MCP and similar protocols dramatically expand model capabilities by exposing tools and execution environments. However, they frequently inflate input context far beyond task requirements. Tool responses often return complete file

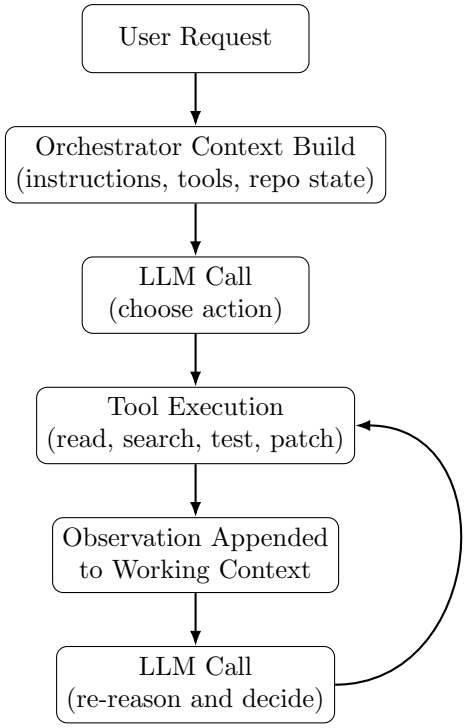

Figure 1: Canonical agentic coding loop. Strength: iterative discovery in uncertain environments. Cost: repeated context growth, latency, and reasoning overhead.

contents when only snippets are relevant, entire directory structures when single paths matter, and comprehensive documentation when specific endpoints suffice.

Anthropic's engineering analyses have documented how MCP interactions can return orders of magnitude more context than necessary [12], exacerbating inefficiencies inherent in agentic systems.

9.2 Context Inflation Dynamics

In agentic workflows with tool access:

1. Model decides to use a tool
2. Tool returns extensive context (often 10,000+ tokens)
3. Model processes context to extract relevant information
4. Only a small fraction influences output
5. Next iteration repeats with fresh tool calls

The cumulative effect is severe context bloat. Projects specifiable in 20,000 tokens routinely consume 200,000+ tokens of input context across iterations, with most never meaningfully influencing outputs.

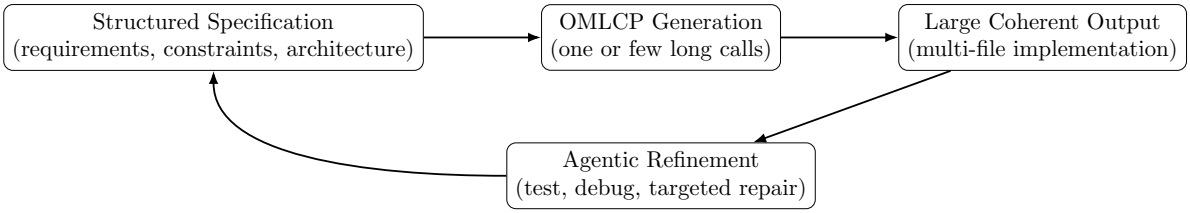

Figure 2: Hybrid execution pattern: OMLCP-first generation (with bounded continuation when needed) followed by agentic refinement. This combines coherence and speed with iterative fault discovery.

9.3 OMLCP's Context Discipline

OMLCP avoids this through strict input curation. Required information is extracted upfront during planning, only relevant excerpts are included in generation prompts, tool integration happens after core generation completes, and external system connections are added deterministically.

10 Implications for Software Engineering Practice

OMLCP's viability suggests several shifts in how we should think about AI-assisted development.

10.1 Redefining Productivity Metrics

Current tools emphasize interaction count and response speed. Under OMLCP, superior productivity is better measured by:

- Ratio of useful output to total tokens consumed
- Coherence and integration success rate
- Time from specification to working artifact
- Maintenance burden of generated code

By these metrics, a single day spent on comprehensive planning followed by one long generation can dramatically outperform a week of iterative development.

10.2 Economic Accessibility and Inclusion

Cost structure directly affects who can adopt AI-assisted software development. Large enterprise deployments demonstrate the scale of spend: OpenHands reports that C3 AI processed approximately 8 billion tokens in two weeks running OpenHands across a small internal user base—a scale of inference spend that is prohibitive for most smaller organizations [11].

Broader ecosystem data similarly shows rapid growth in production token throughput [10]. In contrast, OMLCP's low token intensity shifts capability toward smaller teams, emerging-market developers, public-sector organizations, and research groups operating under tight budget constraints. This is not only an efficiency claim; it is an accessibility claim about who can practically build with frontier models.

10.3 The Role of Planning

OMLCP elevates planning from optional to essential. The workflow assumes clear requirements, explicit architecture, and detailed specifications before generation begins. Interestingly, AI assistance is valuable during planning phases-iterative conversation works well for refining requirements and exploring design alternatives. OMLCP shifts iteration from the implementation phase to the specification phase, where it's more valuable and less expensive.

10.4 Skill Requirements

OMLCP demands a different competency profile from developers. Iterative micro-debugging remains useful, but comparative advantage shifts toward specification quality, architectural abstraction, and disciplined validation planning.

Less important:

- Rapid context switching
- Incremental debugging loops
- Conversational trial-and-error refinement

More important:

- Comprehensive upfront design
- Contract-aligned specification writing
- Architectural decomposition and interface clarity
- Systematic testing and validation planning

A practical adoption barrier is that many developers are not trained to produce dense upfront specifications. In our tests, this burden was reduced through AI-augmented planning: ChatGPT in thinking mode with web access was used to gather source material, compare implementation options, and draft contract-compliant technical specifications before generation [3]. This makes OMLCP more accessible, but still requires human review to ensure requirements are complete and executable.

This shift elevates software engineering practice, emphasizing design quality and verification discipline over reactive implementation churn.

11 Limitations and Boundary Conditions

OMLCP is not universally optimal. Several limitations deserve explicit acknowledgment.

11.1 Requirement Clarity

OMLCP assumes requirements can be specified clearly upfront. For exploratory projects, research implementations, or domains with emerging requirements, this assumption fails. Agentic approaches remain superior when goals must be discovered through experimentation.

11.2 Output Limits

Current APIs impose practical output limits. As of March 2026, Tier A systems include approximately 1M context with 128,000-token output ceilings (for example GPT-5.4 [3]), while many Tier B systems remain near 64,000 output tokens (for example Claude Sonnet 4.6 and Gemini 3.1 Pro) [4, 5]. While sufficient for many applications, larger systems must be decomposed into OMLCP-scale components. Bounded continuation mitigates output-ceiling interruptions but does not remove the underlying context-window and retrieval constraints. As output limits con-

tinue to increase, this constraint will gradually relax.

11.3 Error Recovery

When OMLCP generation produces errors, they cannot be corrected mid-stream. For certain error types-particularly those requiring architectural changes-this can make repair more costly than iterative approaches. Continuations that are poorly constrained can also introduce drift; in practice, this is mitigated by resuming from explicit structural boundaries with stable output constraints. However, empirical observation suggests major failure modes are rare when planning is thorough.

11.4 Observed Failure Cases in Practice

OMLCP can fail in ways that are economically nontrivial. We have observed runs where an early architectural defect made downstream correction more expensive than regeneration, especially when the defect appeared in foundational modules. In one incident, a protocol mismatch in an alternative streaming stack caused punctuation corruption during stream reconstruction; remediation required protocol-aware SSE parsing and a full re-stream because targeted patching consumed more time and tokens than rerunning the generation. These cases underscore that OMLCP performance depends not only on prompting quality, but also on robust generation infrastructure.

11.5 Model Capability Requirements

OMLCP depends on frontier model capabilities, particularly the ability to maintain coherence across very long outputs. Not all models handle extended generation equally well. Research shows that models can struggle to maintain long-range coherence in very long-form generation, with performance degrading as output length grows [13, 14].

11.6 Long-Output Alignment and Prompt Steering

Most frontier models are not explicitly trained to maximize very long-form implementation output by default. In practice, OMLCP often requires explicit steering to avoid premature summarization, placeholder stubs, or deferred implementation language. This steering burden is operationally important: without clear output constraints, models may under-deliver on lower-priority but essential artifacts such as exhaustive tests, migration scaffolding, or full environment setup.

11.7 Continuation Depth and Throughput Variability

Even with strong prompting, many runs terminate well below theoretical output ceilings in practical deployments. We have observed frequent practical ceilings near 60,000 output tokens, alongside occasional high-performance runs with multiple bounded continuations at the 128,000-token limit in a single conversation trajectory. End-to-end completion time is therefore constrained not only by token limits but also by token throughput, which varies materially across model versions and serving stacks.

11.8 Dependency Freshness and Environment Friction

Training-data cutoffs and package ecosystem churn can cause generated dependency selections to be outdated or incompatible. Empirically, first-pass reliability improves when prompts include explicit dependency manifests with verified versions, environment variable contracts, and setup instructions. These controls reduce post-generation integration friction and increase the probability that generated systems run after standard install/build/start commands.

11.9 Cross-Domain Artifacts and Editing Maturity

Long-form generation quality issues are domain-specific. In creative writing workloads, we observe repeated lexical patterns and phrase overuse that require targeted post-processing. In software workloads, large-scale one-shot editing beyond initial generation remains under-explored, though conceptually feasible through full-repository or full-section constrained edit passes.

11.10 External Validity of Current Evidence

Current evidence remains concentrated in OpenAI frontier model testing with high output ceilings. Broader validation across additional model families, including systems with 64,000+ output windows, is required before making strong cross-provider generalizations.

11.11 Claim Tiers and Epistemic Scope

To improve interpretability and avoid over-claiming, OMLCP claims should be read in three tiers:

Tier 1 (Analytically derived): The token-economics model implies that repeated context rehydration plus per-step reasoning overhead produces superlinear cost growth in iterative workflows under standard assumptions.

Tier 2 (Empirically observed): In our field cases, OMLCP-style generation produced materially lower token usage, shorter wall-clock delivery time, and stronger cross-file coherence than comparable iterative workflows.

Tier 3 (Forward-looking hypothesis): As output and context limits expand, the relative advantage of specification-first, long-form generation is likely to increase for well-specified tasks, while exploratory tasks remain better served by iterative methods.

This tiering separates mathematical implications, observed operational results, and strategic forecasts, making the argument easier to test and refine.

12 Future Directions

OMLCP opens several promising research directions:

12.1 Hybrid Approaches

The dichotomy between pure OMLCP and pure agentic workflows is artificial. Hybrid systems could combine OMLCP's coherent generation with targeted iteration where needed. Research could identify optimal decomposition points and when to switch paradigms.

12.2 Automated Planning

The planning phase could itself be systematized. Research into automated specification generation, requirement extraction, and architectural inference could reduce the human burden of comprehensive upfront design.

12.3 Quality Metrics

Better metrics are needed to quantify coherence and consistency in generated artifacts. Automated assessment of architectural alignment, naming consistency, and thematic unity would help validate OMLCP advantages empirically.

12.4 Tooling Innovation

Current development environments are not optimized for OMLCP workflows. Tools that support comprehensive planning, long-form generation monitoring, and systematic post-generation validation could make OMLCP more accessible to working developers.

12.5 Long-Output Optimization Methods

Further work is needed on prompt patterns and training interventions that increase sustained output length without coherence collapse. This includes testing structured anti-summarization directives, continuation-aware output contracts, and fine-tuning strategies specifically for long-form deterministic generation.

12.6 Model Coverage and Multimodal Guidance

Future evaluations should expand across providers and reasoning configurations, including controlled comparisons at different reasoning-effort levels. Image-guided prompting for implementation and design generation is also an open question and should be tested systematically rather than assumed effective.

12.7 Large-Scale Edit Regimes

A key next step is evaluating OMLCP for whole-repository editing workloads, not just greenfield generation. Candidate protocols include constrained full-repo rewrite passes, section-scoped transformations, and hybrid plans that pair large-context edits with deterministic verification.

12.8 Beyond Software Engineering

OMLCP may extend to other long-output domains such as legal frameworks, policy drafting, and government documentation. These domains offer useful stress tests for coherence, traceability, and deterministic structure under strict formatting constraints.

12.9 Ecosystem Incentives and Product Direction

A strategic uncertainty remains: if agentic interaction becomes the dominant product consumption mode, model and platform optimization may disproportionately favor iterative loops over maximal coherent output. Measuring and publishing long-output benchmarks can help maintain research and product focus on this capability class.

13 Conclusion

Output-Maximizing Long-Context Programming represents a fundamental realignment between workflow architecture and model capability. As language models evolved to support dramatically larger output contexts, the tooling ecosystem largely failed to capitalize on this shift, remaining optimized for interaction patterns appropriate to earlier constraints.

The consequences of this misalignment are substantial. Agentic workflows, while effective for small tasks and exploratory work, exhibit severe compounding inefficiencies when scaling to large artifacts. The majority of computational cost is consumed by non-output overhead, especially context reconstruction and coordination, rather than useful output generation. Coherence suffers as global context fragments across iterations. Economic efficiency degrades quadratically with artifact size.

OMLCP inverts this structure. By consolidating planning upfront and generating complete artifacts in single coherent passes, it eliminates the structural sources of inefficiency. Token budgets shift from invisible deliberation to visible output. Coherence is preserved through unbroken attention rather than reconstructed across gaps. Costs scale linearly rather than quadratically, with empirically observed efficiency improvements of 20-60× for large projects.

Our full-stack application case study demonstrates these advantages at scale: 14,431 lines of working code generated for $0.58 in 2 hours 20 minutes total time, representing a 26-52× cost advantage and 4× time advantage over comparable agentic approaches. The application required no architectural refactoring and exhibited the internal consistency predicted by OMLCP theory.

These advantages are not marginal; they represent the difference between economically impractical and readily feasible applications. Importantly, Tier 2 evidence indicates OMLCP is viable today on current frontier models for well-specified tasks. The barrier to adoption is not technical capability but rather workflow adaptation and cultural acceptance of comprehensive upfront planning over reactive iteration.

As output context windows continue to expand, systems optimized for small outputs will become increasingly inefficient relative to those designed for coherent long-form generation. This transition will not happen automatically. Tool builders must recognize the opportunity and design for it. Developers must adapt their working methods to emphasize planning over immediate action. Organizations must update productivity metrics to value efficiency and coherence over interaction count.

The persistence of agentic paradigms reflects historical necessity rather than present-day optimality. Recognizing and exploiting this shift represents a significant opportunity for both research and practice.

References

[1] OpenAI. GPT-3.5 Turbo Model Documentation. https://developers.openai.com/api/docs/models/gpt-3.5-turbo, accessed March 2026.

[2] OpenAI. Introducing GPT-5.2. https://openai.com/index/introducing-gpt-5-2/, December 2025.

[3] OpenAI. GPT-5.4 Model Documentation. https://developers.openai.com/api/docs/models/gpt-5.4, accessed March 2026.

[4] Anthropic. What's new in Claude 4.6. https://platform.claude.com/docs/en/about-claude/models/whats-new-claude-4-6, accessed March 2026.

[5] Google AI for Developers. Gemini 3.1 Pro Preview. https://ai.google.dev/gemini-api/docs/models/gemini-3.1-pro-preview, accessed March 2026.

[6] M. Salim, J. Latendresse, S. Khatoonabadi, E. Shihab. Tokenomics: Quantifying Where Tokens Are Used in Agentic Software Engineering. MSR 2026. https://arxiv.org/abs/2601.14470, 2026.

[7] Y. Ren, Y. Lin, B. Li, W. Wang, R. Cam, N. Kerdkham, R. Yang, J. Shang, W. Wang. How Do Coding Agents Spend Your Money? Analyzing and Predicting Token Consumptions in Agentic Coding Tasks. ICLR 2026 (withdrawn submission). https://openreview.net/forum?id=1bUeVB3fov, 2025.

[8] W. Hu, R. Pi, W. Wang, Y. Chen, W. Huang, Y. Lin, J. Shang, W. Wang. SWE-Pruner: Self-Adaptive Context Pruning for Coding Agents. https://arxiv.org/abs/2601.16746, 2026.

[9] P. Dannhauer, M. J. Ger, O. O'Brien, T. Toh, C. Canca, S. J. Wang, E. P. Xing. SWE-EVO: Benchmarking Coding Agents in Long-Horizon Software Evolution. https://arxiv.org/abs/2512.18470, 2025.

[10] M. Aubakirova et al. State of AI: An Empirical 100 Trillion Token Study with OpenRouter. https://arxiv.org/abs/2601.10088, 2026.

[11] OpenHands. Eight billion tokens in two weeks: How C3 AI brings agentic coding to every team with OpenHands. https://openhands.dev/blog/eight-billion-tokens-in-two-weeks-how-c3-ai-brings-agentic-coding-to-every-team-with-openhands, December 2025.

[12] Anthropic. Code Execution with MCP. https://www.anthropic.com/engineering/code-execution-with-mcp, 2024.

[13] N. Elhage, E. C. Caballero, C. Olah. Context Rot: How Increasing Input Tokens Impacts LLM Performance. https://research.trychroma.com/context-rot, 2025.

[14] N. Ye, K. Hwang, W. Yu, X. Du, W. Zhuang, N. Jiang, X. Ren. LongProc: A Benchmark for Long-Context Large Language Models. https://arxiv.org/abs/2501.05414, 2025.

[15] S. Yao et al. ReAct: Synergizing Reasoning and Acting in Language Models. https://arxiv.org/abs/2210.03629, 2023.

[16] X. Li et al. Chain of Agents: Large Language Models Collaborating on Long-Context Tasks. https://arxiv.org/abs/2406.02818, 2024.

[17] Q. Li, S. Wang, Y. Chai, Y. Shao, X. Zhang, X. Dai, X. F. Zhang, Y. Wang, J.-R. Wen, Y. Ban. CoDA: A Context-Decoupled Hierarchical Agent with Reinforcement Learning. https://arxiv.org/abs/2512.12716, 2025.

[18] M. Hajizadeh, H. Vahabi, M. Hajiabadi, M. Rezagholizadeh, A. Ghodsi. Emulating Retrieval Augmented Generation via Prompt

Engineering for Enhanced Long Context Comprehension in LLMs. https://arxiv.org/abs/2502.12462, 2025.

[19] Weaviate. Context Engineering: LLM Memory and Retrieval for AI Applications. https://weaviate.io/blog/context-engineering, 2025.

[20] T. Schick et al. Toolformer: Language Models Can Teach Themselves to Use Tools. https://arxiv.org/abs/2302.04761, 2023.

[21] N. Shinn et al. Reflexion: Language Agents with Verbal Reinforcement Learning. https://arxiv.org/abs/2303.11366, 2023.

[22] X. Wang et al. Self-Consistency Improves Chain of Thought Reasoning in Language Models. https://arxiv.org/abs/2203.11171, 2022.

[23] N. F. Liu et al. Lost in the Middle: How Language Models Use Long Contexts. https://arxiv.org/abs/2307.03172, 2023.

[24] J. Yang et al. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. https://arxiv.org/abs/2405.15793, 2024.